PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA

FACULTAD DE CIENCIAS E INGENIERÍA

ESCUELA DE INGENIERÍA EN COMPUTACIÓN Y TELECOMUNICACIONES

DESARROLLO DE APLICACIONES MÓVILES

PROYECTO FINAL

Desarrollo de una Pokédex con Flutter, GraphQL y Arquitectura Offline-First

Diseño de una aplicación móvil resiliente mediante arquitectura limpia, persistencia local y sincronización inteligente de datos.

ENFOQUE ARQUITECTÓNICO

Offline-First + Online-Preferred

FUENTE DE DATOS

PokeAPI — servicio GraphQL

DOCENTE

Freddy Peña · fr.pena@ce.pucmm.edu.do

MODALIDAD

Equipos de máximo dos estudiantes

EVALUACIÓN

30-20 puntos · presentación presencial obligatoria

PERÍODO ACADÉMICO · FECHA DE ENTREGA

Contenido

1. Descripción general del proyecto
2. Objetivo general
3. Objetivos específicos
4. Alcance y condiciones generales
5. Principios arquitectónicos
6. Arquitectura Offline-First
7. Gestión de conectividad e indicador global
8. Arquitectura de software
9. Requerimientos funcionales
10. Requerimientos no funcionales
11. Persistencia y sincronización
12. Diagramas de arquitectura y comportamiento
13. Pruebas de aceptación
14. Entregables y documentación
15. Presentación y defensa técnica
16. Rúbrica de evaluación
17. Criterios de cumplimiento
18. Referencias técnicas

Convenciones del documento

`OF · IND`	Requisitos del modelo Offline-First y del indicador global de conectividad.
`RF-xxx-nn`	Requerimientos funcionales, agrupados por funcionalidad (CAT, DET, FAV, FIL, TRV, COM).
`RNF-xxx-nn`	Requerimientos no funcionales (REN, RES, MAN, ACC, CAL).
`PS · ENT`	Decisiones de persistencia y sincronización, y entregables.
`PA-nn`	Escenarios de pruebas de aceptación verificables durante la demostración.
`D1 … D6`	Diagramas de referencia, ubicados junto a la sección que explican.

Cada requisito indica su nivel de exigencia: **Obligatorio**, **Recomendado** u **Opcional** (definidos en §4.3).

1 Descripción general del proyecto

El proyecto final consiste en el diseño, desarrollo y defensa de una Pokédex móvil multiplataforma construida con Flutter, que consume la API GraphQL de PokeAPI y opera con una arquitectura Offline-First. La aplicación debe permitir consultar el catálogo, el detalle de cada Pokémon, gestionar favoritos y jugar una trivia, tanto con conexión a Internet como sin ella.

El proyecto está dirigido a estudiantes con experiencia previa en Flutter y no tiene carácter introductorio. Su propósito es evaluar competencias de ingeniería de software aplicadas al desarrollo móvil: diseño de arquitecturas mantenibles, separación de responsabilidades, integración de servicios GraphQL, persistencia estructurada, gestión de conectividad y sincronización, resiliencia ante fallos, gestión de estado y documentación técnica.

> **PRINCIPIO FUNDAMENTAL**
>
> La Pokédex es el dominio funcional utilizado para demostrar las competencias de arquitectura móvil. La cantidad de pantallas, animaciones o elementos decorativos no tiene mayor peso que la calidad de la solución técnica.

Este enunciado actualiza la versión anterior del proyecto. Se conservan los requisitos funcionales de la Pokédex que siguen siendo pertinentes y se incorporan como eje central el modelo Offline-First, el indicador global de conectividad, los requerimientos no funcionales, las pruebas de aceptación y una rúbrica basada en evidencias. El documento define contratos, restricciones y resultados esperados; las decisiones de implementación corresponden a cada equipo, que deberá justificarlas.

2 Objetivo general

Diseñar, desarrollar y presentar una aplicación móvil multiplataforma utilizando Flutter y GraphQL, fundamentada en una arquitectura Offline-First con comportamiento Online-Preferred, que permita consultar información de Pokémon de manera continua, tanto con conexión a Internet como sin ella, mediante mecanismos robustos de persistencia local, sincronización automática, gestión de estado y recuperación ante fallos.

La aplicación deberá demostrar buenas prácticas de ingeniería de software, arquitectura limpia, accesibilidad, experiencia de usuario y documentación técnica.

3 Objetivos específicos

OE-01	Aplicar los principios de Clean Architecture en Flutter mediante una organización por funcionalidades con separación de capas.
OE-02	Integrar la API GraphQL de PokeAPI con consultas eficientes, paginación y manejo diferenciado de errores.
OE-03	Diseñar e implementar una persistencia local estructurada que funcione como fuente de verdad observable de la interfaz.
OE-04	Implementar una sincronización resiliente, con reintentos controlados, prevención de duplicados y recuperación ante interrupciones.
OE-05	Gestionar la conectividad y la disponibilidad real del servicio remoto, y comunicarlas mediante un indicador global.
OE-06	Aplicar una gestión de estado estructurada y reactiva que modele explícitamente los estados de carga, éxito, error y ausencia de datos.
OE-07	Desarrollar una experiencia de usuario fluida y accesible, con soporte de internacionalización y elementos de gamificación.
OE-08	Documentar y versionar la solución en GitHub con buenas prácticas de ingeniería.

4 Alcance y condiciones generales

4.1 Fuente de datos

La información de Pokémon se obtiene de la API GraphQL de PokeAPI. Documentación: https://pokeapi.co. Consola GraphQL: https://graphql.pokeapi.co/v1beta2/console.

PokeAPI es una fuente remota principalmente de lectura. No se exige sincronización bidireccional con PokeAPI. Los favoritos, puntuaciones y logros pueden permanecer exclusivamente en almacenamiento local.

4.2 Dentro y fuera del alcance

DENTRO DEL ALCANCE

- Lectura de datos de PokeAPI mediante GraphQL.
- Persistencia local estructurada como fuente de verdad.
- Sincronización unidireccional (remoto → local).
- Datos propios del usuario almacenados localmente.
- Indicador global de conectividad.
- Documentación técnica.

FUERA DEL ALCANCE

- Escritura de datos en PokeAPI.
- Sincronización bidireccional o resolución de conflictos con un servidor.
- Descarga obligatoria del catálogo completo en la primera ejecución.
- Backend propio, autenticación o cuentas de usuario.

4.3 Niveles de exigencia

Obligatorio	Debe implementarse y demostrarse. Se considera en el cálculo de cumplimiento mínimo (§17).
Recomendado	Práctica o tecnología sugerida. Su ausencia no penaliza si se presenta una alternativa justificada; su implementación correcta se valora en la rúbrica.
Opcional	Funcionalidad adicional. Puede valorarse en el criterio de gamificación y creatividad; su ausencia no penaliza.

4.4 Condiciones generales

- Equipos de máximo dos estudiantes.
- Código fuente en un repositorio GitHub, público o privado compartido con el docente.
- Presentación presencial obligatoria con participación de ambos integrantes (§15).
- Evaluación individual y grupal basada en contribuciones reales verificables (commits, pull requests, issues).
- Cumplimiento mínimo del 80% (§17).

5 Principios arquitectónicos

Los siguientes principios orientan todas las decisiones de diseño y sirven como referencia común para la defensa técnica.

P-01 **Fuente de verdad local.** La interfaz observa la persistencia local; las respuestas remotas actualizan esa persistencia y nunca se consumen directamente como estado de pantalla.

P-02	**Online-Preferred.** Cuando el servicio está disponible, la aplicación prioriza obtener información actualizada, sin bloquear la navegación.
P-03	**Separación de responsabilidades.** Cada capa y cada funcionalidad tienen responsabilidades delimitadas y dependencias explícitas.
P-04	**Regla de dependencia.** Las dependencias apuntan hacia el dominio; el dominio no conoce detalles de infraestructura ni de interfaz.
P-05	**Degradación segura.** Los fallos remotos degradan la funcionalidad de forma controlada, sin perder datos locales válidos ni interrumpir la aplicación.
P-06	**Transparencia del estado.** El usuario conoce en todo momento el estado de la conexión y el origen y la vigencia de los datos que visualiza.
P-07	**Verificabilidad.** Cada decisión relevante se respalda con evidencia demostrable o un registro de decisión (ADR).

6 Arquitectura Offline-First

Este es el componente arquitectónico más importante del proyecto. La aplicación debe implementar un modelo **Offline-First + Online-Preferred**: la interfaz siempre se construye a partir de los datos persistidos localmente y, cuando el servicio remoto está disponible, la aplicación prioriza obtener información actualizada mediante GraphQL y almacenarla en la persistencia local.

6.1 Funcionamiento con Internet

CÓDIGO	REQUISITO	NIVEL
OF-01	Con el servicio remoto disponible, la aplicación consulta información actualizada mediante GraphQL.	Obligatorio
OF-02	Los datos obtenidos se persisten localmente y la interfaz refleja la información actualizada a través de la persistencia.	Obligatorio
OF-03	La aplicación nunca bloquea la interfaz. Desde la primera ejecución muestra la interfaz de inmediato (estado de carga o vacío) y permite navegar mientras la sincronización ocurre en segundo plano.	Obligatorio
OF-04	Los datos locales se conservan ante errores remotos; una respuesta fallida o inválida no elimina ni sobrescribe información válida.	Obligatorio

6.2 Funcionamiento sin Internet

CÓDIGO	REQUISITO	NIVEL
OF-05	Sin conexión, la aplicación continúa funcionando con los datos previamente almacenados; la navegación no depende de solicitudes remotas.	Obligatorio
OF-06	Los usuarios pueden consultar los Pokémon descargados; las búsquedas y filtros funcionan sobre los datos locales disponibles.	Obligatorio
OF-07	Los favoritos continúan operativos (consulta, alta y baja).	Obligatorio
OF-08	Las imágenes previamente almacenadas pueden visualizarse; las no disponibles se sustituyen por un recurso de respaldo.	Obligatorio
OF-09	La trivia funciona utilizando datos locales suficientes.	Obligatorio
OF-10	La aplicación conserva su funcionamiento después de cerrarse y volver a abrirse sin conexión.	Obligatorio

No se exige que el catálogo completo esté descargado desde la primera ejecución. Cada equipo debe definir y documentar su estrategia de disponibilidad local (§11, PS-02).

6.3 Recuperación de conectividad

CÓDIGO	REQUISITO	NIVEL
OF-11	Al recuperarse la conectividad, la aplicación detecta la disponibilidad real del servicio remoto, no solo la presencia de red.	Obligatorio
OF-12	Reanuda las operaciones de actualización correspondientes y actualiza la persistencia local.	Obligatorio
OF-13	No genera registros duplicados ni pierde información local válida.	Obligatorio
OF-14	No bloquea la experiencia del usuario durante la recuperación.	Obligatorio
OF-15	Gestiona errores de sincronización mediante reintentos controlados (ver RNF-RES-02).	Obligatorio

6.4 Fuente de verdad y autoridad de los datos

CÓDIGO	REQUISITO	NIVEL
OF-16	La persistencia local es la fuente de lectura observable para la interfaz; PokeAPI es la fuente autoritativa de los datos del catálogo.	Obligatorio
OF-17	Los widgets no dependen directamente de las respuestas GraphQL.	Obligatorio
OF-18	El acceso a datos se realiza mediante repositorios y abstracciones apropiadas.	Obligatorio
OF-19	Sincronización obligatoria remoto → local: si un dato del catálogo se modifica en la base local, la aplicación detecta la diferencia y lo restaura con la versión de PokeAPI. En caso de conflicto, prevalece siempre el dato remoto.	Obligatorio
OF-20	La comprobación se ejecuta, como mínimo, al iniciar la aplicación, al recuperar la conectividad, al abrir el detalle de un Pokémon y de forma periódica mientras el servicio esté disponible.	Obligatorio
OF-21	La restauración se aplica solo a datos del catálogo. Favoritos, puntuaciones y logros son datos del usuario sin equivalente remoto y no se sobrescriben.	Obligatorio

La aplicación puede consultar prioritariamente el servicio remoto cuando esté disponible, pero la actualización de la interfaz debe mantenerse consistente con los datos persistidos localmente: la respuesta remota se valida, se almacena y la interfaz recibe el cambio a través de la observación de la base local.

6.5 Offline-First frente a caché HTTP

Una aplicación que solo almacena respuestas de red en caché no satisface este proyecto. La tabla siguiente resume la diferencia que se evaluará.

ASPECTO	OFFLINE-FIRST (EXIGIDO)	CACHÉ HTTP / DE RESPUESTAS
Fuente de la UI	Base local observable.	Respuesta de red, eventualmente servida desde caché.
Modelo de datos	Estructurado y consultable: entidades, relaciones, índices.	Respuestas opacas indexadas por URL o por consulta.
Consultas nuevas sin red	Búsquedas, filtros y ordenaciones funcionan sobre los datos locales.	Solo se responde a solicitudes idénticas ya almacenadas.
Datos del usuario	Favoritos y trivia forman parte del mismo modelo local.	Fuera del alcance de la caché.
Vigencia	Política de actualización explícita, con registro de última sincronización.	Gobernada por TTL o cabeceras; puede descartar datos válidos.
Reconexión	Sincronización coordinada, con reintentos y control de concurrencia.	Nueva solicitud sin coordinación.

La caché normalizada de un cliente GraphQL puede complementar la solución, pero no sustituye la persistencia estructurada exigida en OF-16.

6.6 Flujo de referencia

El Diagrama 1 describe el comportamiento esperado desde el inicio de la aplicación hasta la reconexión. El Diagrama 2 detalla la interacción entre componentes en los escenarios online, offline y de error remoto.

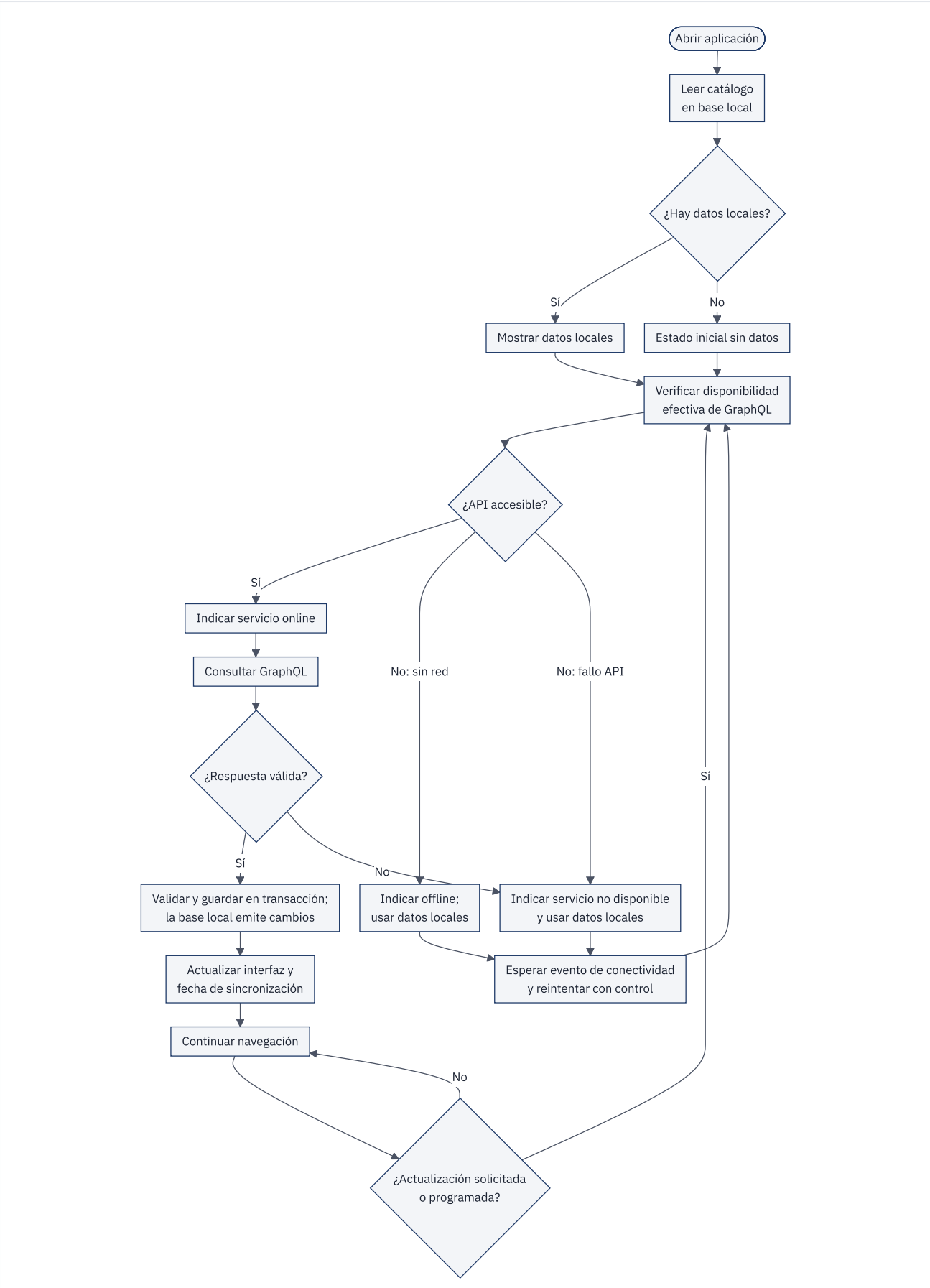


Diagrama 1 — Flujo Online / Offline. Sin datos precargados, la primera ejecución offline presenta un estado vacío útil. Los reintentos tienen límites y backoff.

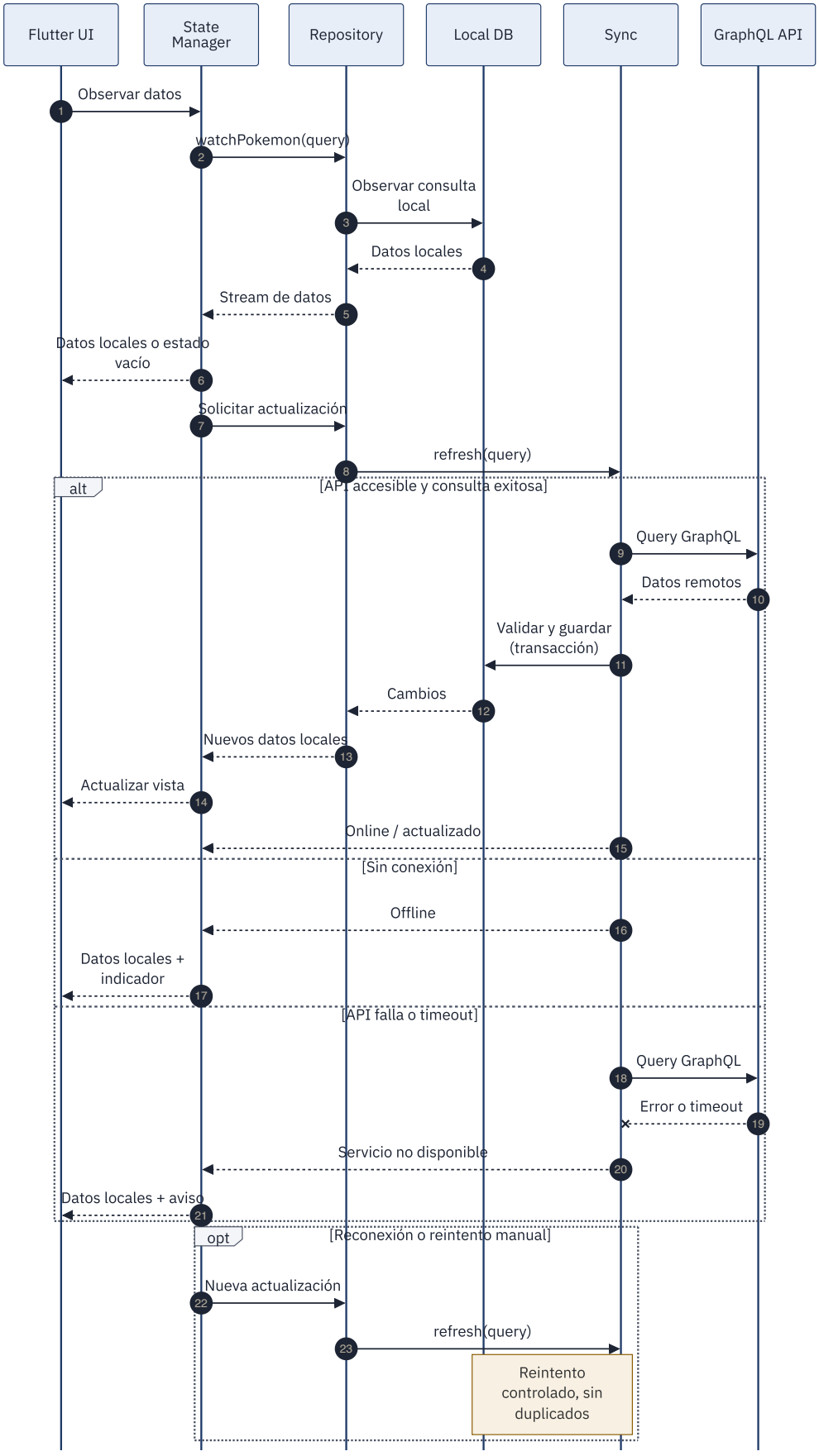


Diagrama 2 — Secuencia de consulta y recuperación. La interfaz mantiene la suscripción a la base local durante todo el proceso. Los datos visibles pueden seguir siendo locales aunque la conectividad ya se haya restablecido; «hay red» no equivale a «los datos están actualizados».

7 Gestión de conectividad e indicador global

La aplicación debe incorporar un indicador global, fijo, visible y reactivo que informe al usuario sobre el estado de conexión y el origen de los datos. Este requisito es obligatorio.

7.1 Estados requeridos

ESTADO	CONDICIÓN	COMPORTAMIENTO ESPERADO
● **ONLINE**	Servicio remoto accesible; consultas GraphQL disponibles.	Se identifican los datos remotos actualizados.
○ **OFFLINE**	Servicio inaccesible por ausencia de conectividad.	Funcionamiento con información local e identificación visual clara del modo offline.
▲ **SERVICIO NO DISPONIBLE**	Existe conectividad de red, pero la API no responde correctamente.	Uso de datos locales. El indicador no presenta falsamente un estado online.
↻ **SINCRONIZANDO**	La aplicación está actualizando la información local.	El usuario puede seguir navegando; retroalimentación visual no intrusiva.

7.2 Reglas del indicador

CÓDIGO	REQUISITO	NIVEL
`IND-01`	El indicador está presente en todas las pantallas principales, en una posición fija y consistente.	Obligatorio
`IND-02`	Determina el estado a partir de la disponibilidad real del servicio GraphQL, no exclusivamente de la detección de Wi-Fi o datos móviles.	Obligatorio
`IND-03`	Se actualiza automáticamente ante cambios de red, de disponibilidad del servicio y de actividad de sincronización.	Obligatorio
`IND-04`	Distingue el estado de conexión de la procedencia (local o remota) y la vigencia (última actualización) de los datos visibles.	Obligatorio
`IND-05`	Es accesible y comprensible sin depender exclusivamente del color: combina texto e icono y expone una etiqueta Semantics.	Obligatorio
`IND-06`	Evita cambios visuales molestos o bloqueantes, incluidos parpadeos ante variaciones breves de conectividad.	Obligatorio
`IND-07`	Durante la sincronización ofrece retroalimentación no intrusiva y permite continuar navegando.	Obligatorio

7.3 Mockup de referencia

La Figura 1 ilustra el indicador en sus cuatro estados sobre la pantalla de catálogo. Es una referencia conceptual de jerarquía de información; el diseño visual, la posición y los textos concretos son decisión de cada equipo, siempre que se cumplan IND-01 a IND-07.

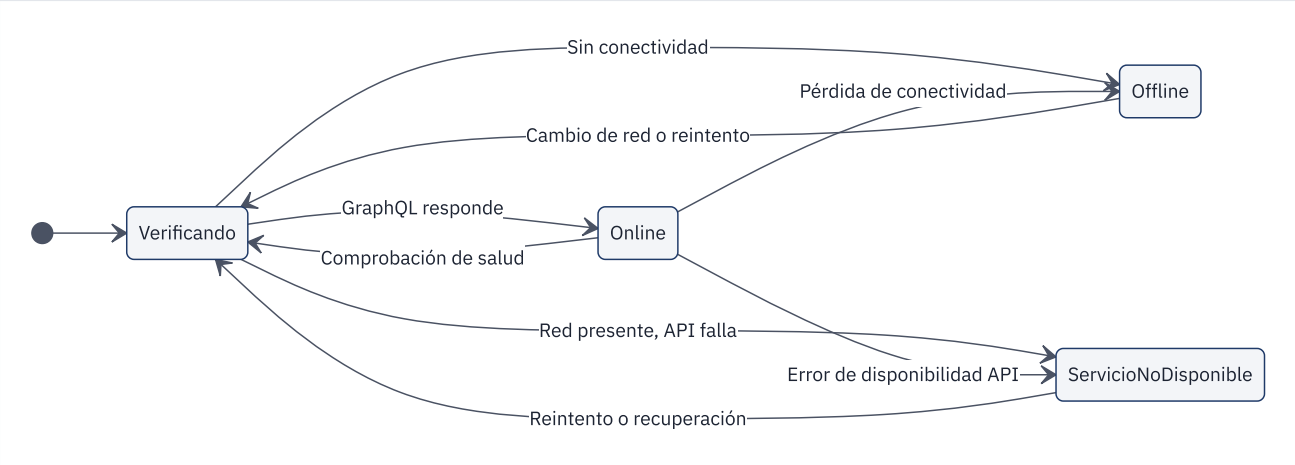


Figura 1 — Indicador global en sus cuatro estados (conceptual). Cada estado combina símbolo, texto y color; la segunda línea comunica el origen y la vigencia de los datos, independiente del estado de conexión.

7.4 Modelo de estados

La disponibilidad del servicio y la sincronización son dos dimensiones independientes. El estado de sincronización puede coexistir conceptualmente con la disponibilidad de red (por ejemplo, «online y sincronizando»), por lo que el modelo interno debe representarlas por separado y evitar estados contradictorios, como declarar simultáneamente «offline» y «API accesible». El indicador compone ambas dimensiones junto con el origen y la frescura de los datos.

3.1 DISPONIBILIDAD DEL SERVICIO



3.2 ESTADO DE SINCRONIZACIÓN

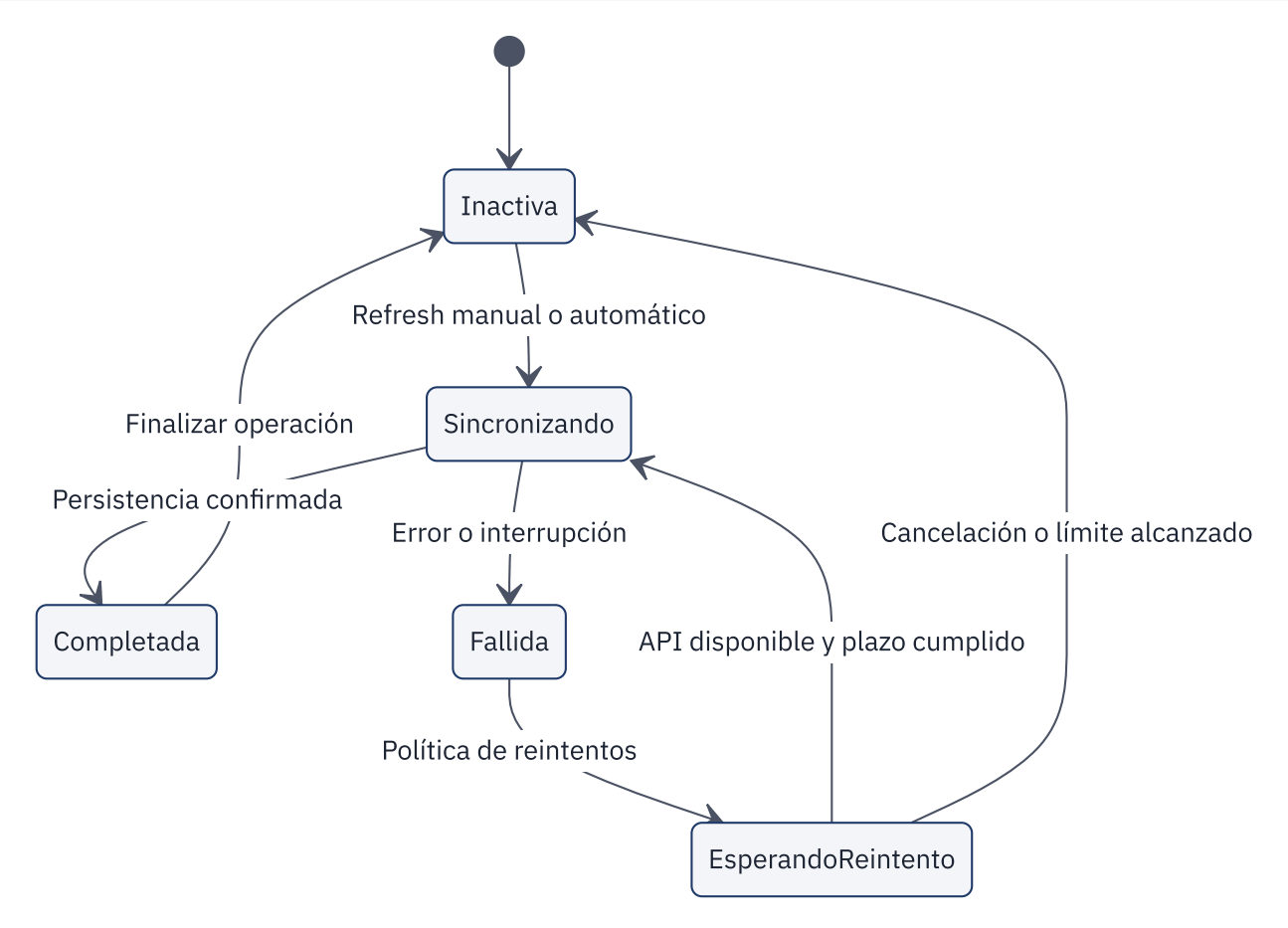


Diagrama 3 — Máquinas de estados de conectividad y sincronización. El indicador muestra al menos el estado del servicio y, cuando aplique, la actividad de sincronización; además aclara si el contenido procede de la base local y cuándo se actualizó.

8 Arquitectura de software

Se exige una arquitectura organizada por funcionalidades, con separación clara de responsabilidades según los principios de Clean Architecture. Las dependencias apuntan hacia el dominio.

8.1 Capas y responsabilidades

CAPA	RESPONSABILIDADES MÍNIMAS	RESTRICCIONES
Presentation	Widgets, pantallas y navegación; gestión de estado; estados de carga, éxito, error y ausencia de información; indicador de conectividad; interacciones y retroalimentación visual.	No accede directamente al cliente GraphQL ni a la base de datos.
Domain	Entidades del dominio; casos de uso cuando correspondan; contratos de repositorios; reglas de negocio.	Independiente de Flutter, del cliente GraphQL y de la persistencia.
Data	Implementación de repositorios; fuentes de datos locales y remotas; DTOs y mapeadores; persistencia; coordinación de sincronización; gestión de errores de infraestructura.	Traduce errores de infraestructura a fallos del dominio; no expone DTOs a la presentación.

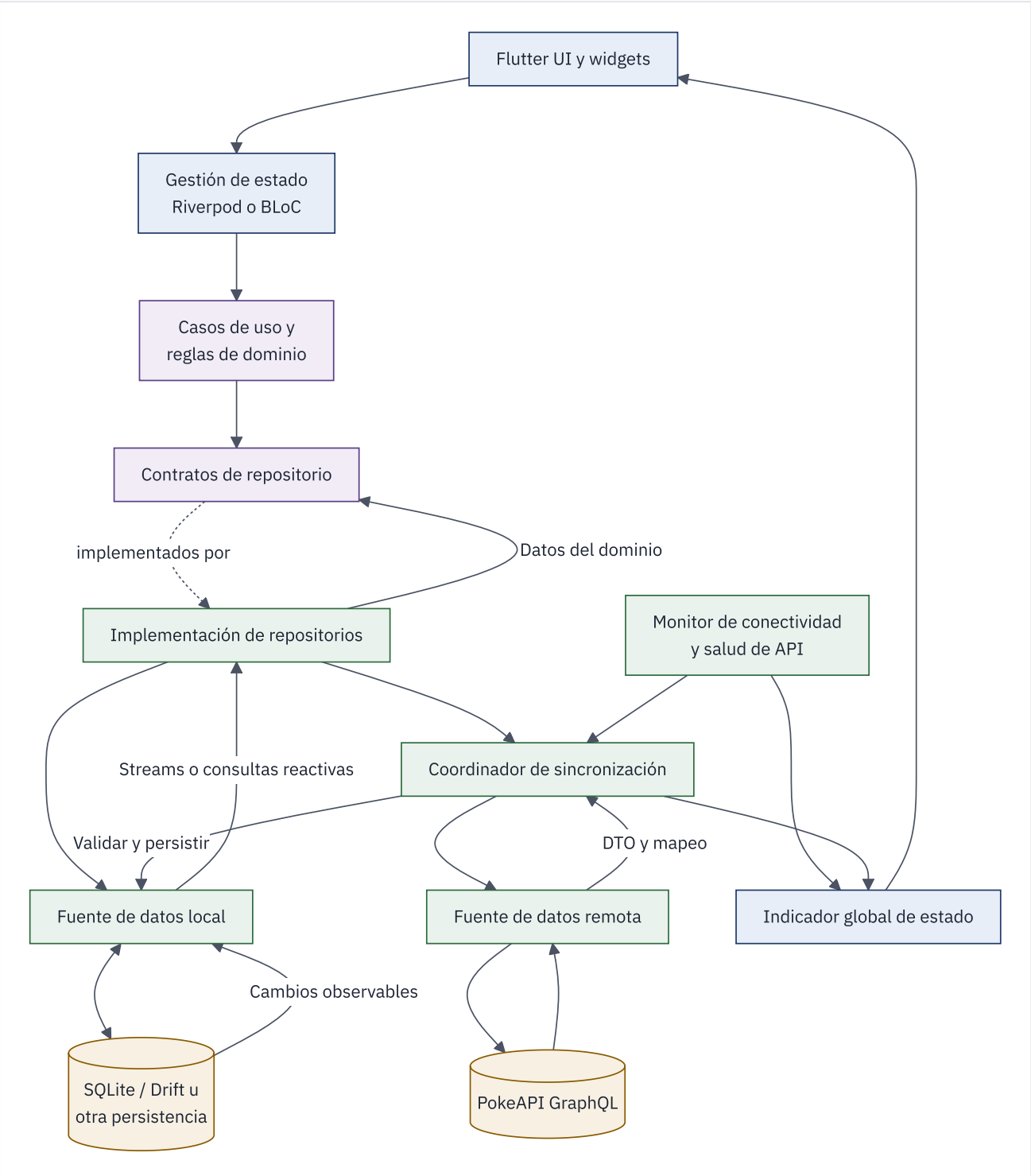


Diagrama 4 — Arquitectura general. El flujo de lectura de la interfaz pasa por la fuente local, que es la fuente de verdad observable. La fuente remota alimenta la persistencia mediante el repositorio y el coordinador de sincronización. El monitor comprueba la disponibilidad efectiva de la API, no solo la presencia de red.

8.2 Organización del proyecto

El Diagrama 5 presenta una organización de referencia por funcionalidades con capas internas y componentes transversales. Es conceptual: se evalúa la separación de responsabilidades, no la coincidencia literal de carpetas. Los componentes transversales (`core/`) no deben

convertirse en un depósito de lógica específica de cada funcionalidad.

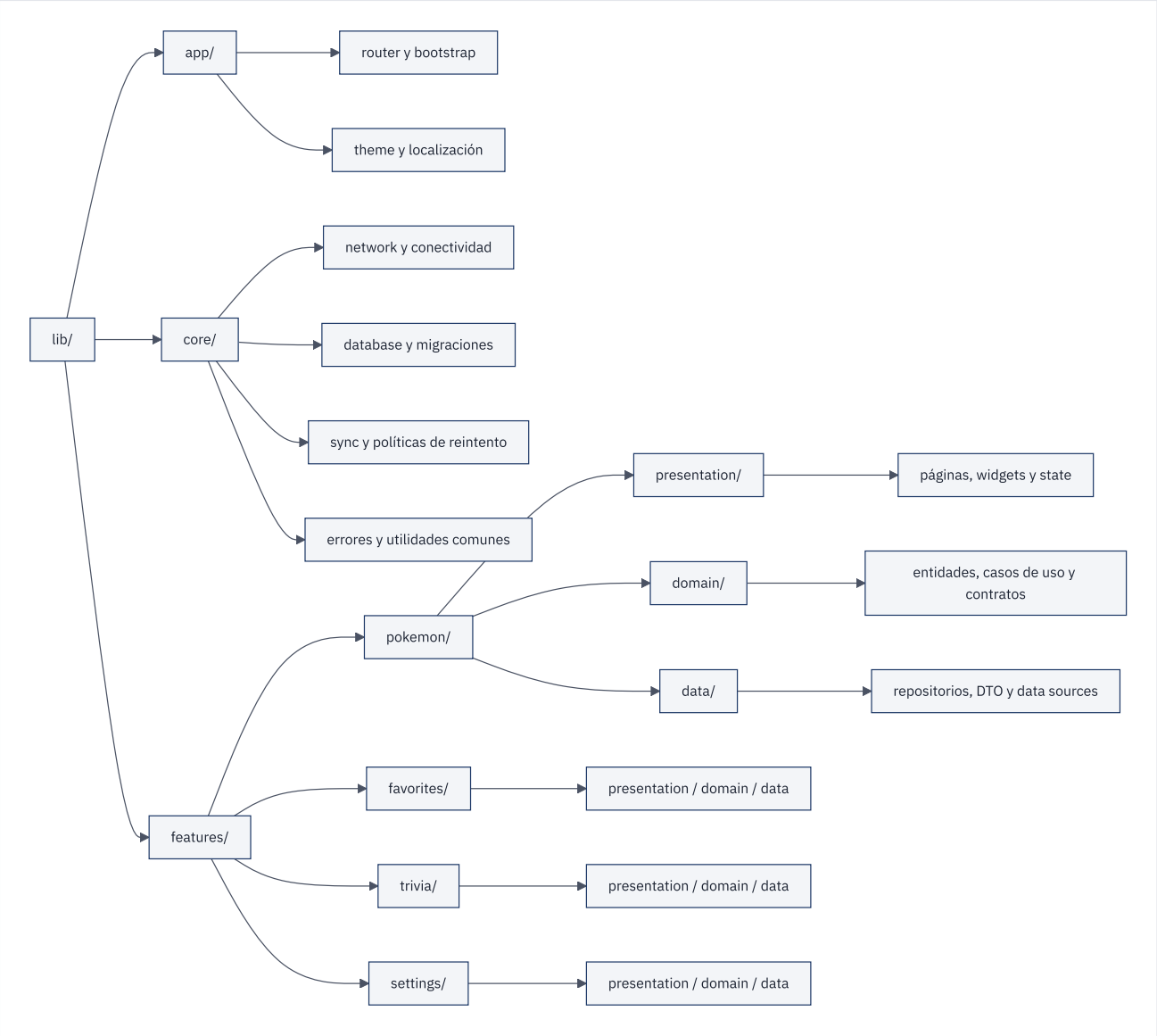


Diagrama 5 — Estructura del proyecto por funcionalidades y capas. Nombres orientativos. La capa de dominio no depende de widgets, del cliente GraphQL ni de la implementación de la base de datos.

8.3 Tecnologías

OBLIGATORIAS

- Flutter y Dart.
- GraphQL (PokeAPI).
- Persistencia local estructurada.
- Gestión de estado estructurada.

RECOMENDADAS

- Riverpod o BLoC para gestión de estado.
- SQLite mediante Drift para persistencia.
- Un cliente GraphQL para Dart.
- Inyección de dependencias.

Se aceptan alternativas técnicamente justificadas. La elección de cada dependencia relevante debe documentarse (RNF-MAN-04) y las decisiones de mayor impacto deben registrarse como ADR (§14).

9 Requerimientos funcionales

Se conservan y reorganizan los requisitos de la versión original. Los criterios de aceptación específicos se incluyen en la descripción de cada requisito.

9.1 Catálogo de Pokémon

CÓDIGO	REQUISITO	NIVEL
RF-CAT-01	Listado de Pokémon con nombre, número de Pokédex nacional, imagen y tipos.	Obligatorio
RF-CAT-02	Paginación del catálogo. La estrategia (offset, cursor u otra) se documenta y justifica según las capacidades de la API.	Obligatorio
RF-CAT-03	Búsqueda por nombre con debounce de 300 a 500 ms.	Obligatorio
RF-CAT-04	Estados diferenciados de carga, éxito, ausencia de datos y error; la pantalla de error muestra un mensaje comprensible y la opción «Reintentar».	Obligatorio
RF-CAT-05	Consulta offline de los datos disponibles localmente.	Obligatorio
RF-CAT-06	Diseño responsivo para distintos tamaños de pantalla. El lenguaje visual es libre.	Obligatorio
RF-CAT-07	Temas dinámicos (color derivado del sistema o del Pokémon).	Recomendado
RF-CAT-08	Uso de Material Design 3 como sistema de diseño.	Opcional

9.2 Detalle de Pokémon

CÓDIGO	REQUISITO	NIVEL
RF-DET-01	Identidad: nombre, número de Pokédex nacional, tipos (1–2) y official-artwork. Si hay dos tipos se muestran ambos; si falta la imagen oficial se usa el sprite como respaldo.	Obligatorio
RF-DET-02	Estadísticas base (HP, Atk, Def, SpA, SpD, Spe) y total, en gráfico radar o de barras. Cada estadística muestra valor y porcentaje; el total es visible.	Obligatorio
RF-DET-03	Habilidades: nombre, indicador «Oculta» cuando isHidden es verdadero y efecto resumido de 140 a 160 caracteres como máximo.	Obligatorio

RF-DET-04	Evoluciones: línea o árbol con soporte de ramas y trigger principal (nivel, objeto, intercambio, amistad, hora). Sin evoluciones, se muestra el estado vacío «No evoluciona».	Obligatorio
RF-DET-05	Movimientos: por defecto, solo el version group seleccionado y el método level-up; filtros por método (level-up, TM, Tutor, Egg); orden por nivel o nombre; lista virtualizada o por secciones que muestre al menos 20 elementos sin degradación perceptible.	Obligatorio
RF-DET-06	Matchups de daño por tipo: debilidades, resistencias e inmunidades (×4, ×2, ×0.5, ×0.25, ×0).	Obligatorio
RF-DET-07	Metadatos: peso, altura y egg groups.	Obligatorio
RF-DET-08	Variantes y formas (Alolan, Galarian, etc.) mediante un selector que actualiza sprite, tipos y movimientos, cuando la especie las tenga.	Obligatorio
RF-DET-09	Alternancia de visualización shiny cuando existan recursos.	Obligatorio
RF-DET-10	Acciones: favorito con animación, compartir y acceso a regiones o juegos cuando aplique.	Obligatorio

9.3 Favoritos

CÓDIGO	REQUISITO	NIVEL
RF-FAV-01	Agregar y eliminar favoritos con persistencia local.	Obligatorio
RF-FAV-02	Pantalla dedicada de favoritos.	Obligatorio
RF-FAV-03	Disponibilidad offline de los favoritos y de sus datos asociados.	Obligatorio
RF-FAV-04	Actualización reactiva: el cambio se refleja en catálogo y detalle sin recarga manual. La persistencia del cambio debe completarse en menos de 150 ms en el dispositivo de demostración.	Obligatorio

9.4 Búsqueda, filtros y ordenación

CÓDIGO	REQUISITO	NIVEL
RF-FIL-01	Filtros por tipo y por generación.	Obligatorio
RF-FIL-02	Filtros adicionales por región o habilidad.	Recomendado
RF-FIL-03	Ordenación por nombre, número y al menos un criterio derivado de estadísticas (por ejemplo, total base).	Obligatorio
RF-FIL-04	Conservación de filtros, búsqueda y ordenación entre sesiones.	Obligatorio

RF-FIL-05	Funcionamiento offline sobre el catálogo disponible; si el catálogo local es parcial, la interfaz lo comunica.	Obligatorio

9.5 Trivia: «¿Quién es este Pokémon?»

CÓDIGO	REQUISITO	NIVEL
RF-TRV-01	Mostrar la silueta de un Pokémon y solicitar al usuario que identifique su nombre.	Obligatorio
RF-TRV-02	Sistema de puntuación y tiempo límite por pregunta o partida.	Obligatorio
RF-TRV-03	Ranking local y persistencia de resultados.	Obligatorio
RF-TRV-04	Logros visuales desbloqueados al alcanzar umbrales definidos y documentados por el equipo.	Obligatorio
RF-TRV-05	Ejecución offline cuando existan datos locales suficientes (umbral definido por el equipo); en caso contrario, mensaje explicativo.	Obligatorio
RF-TRV-06	Disponible en español e inglés.	Obligatorio

9.6 Funcionalidades complementarias

Se clasifican por complejidad estimada de implementación (baja, media, alta) para orientar la planificación del equipo.

CÓDIGO	REQUISITO	COMPL.	NIVEL
RF-COM-01	Generar una «Pokémon Card» y compartirla como imagen.	Media	Obligatorio
RF-COM-02	Animaciones Hero entre catálogo y detalle.	Baja	Obligatorio
RF-COM-03	Microinteracciones y transiciones suaves, incluida la animación al agregar o quitar favoritos.	Baja	Obligatorio
RF-COM-04	Accesibilidad según RNF-ACC.	Media	Obligatorio
RF-COM-05	Internacionalización español/inglés en toda la aplicación (la trivia es obligatoria según RF-TRV-06).	Media	Recomendado
RF-COM-06	Temas claro y oscuro.	Baja	Recomendado
RF-COM-07	Mapa ilustrado interactivo de regiones o juegos en que aparece cada Pokémon.	Alta	Recomendado
RF-COM-08	Onboarding introductorio animado al iniciar la aplicación.	Baja	Opcional

Las regiones de Pokémon son ficticias. No se deben utilizar mapas geográficos convencionales (por ejemplo, teselas de mapas del mundo real) para representarlas sin una justificación funcional.

10 Requerimientos no funcionales

Los criterios cuantitativos se expresan de forma verificable, con el método de verificación indicado. Donde no existe un umbral objetivo, se exige que el equipo defina y documente el valor elegido.

10.1 Rendimiento

CÓDIGO	REQUISITO	NIVEL
RNF-REN-01	Listas construidas de forma diferida (lazy) o virtualizada; no se construyen todos los elementos del catálogo a la vez.	Obligatorio
RNF-REN-02	Consultas locales de búsqueda, filtro y orden optimizadas; los índices utilizados se documentan.	Obligatorio
RNF-REN-03	La sincronización no bloquea el hilo de interfaz; el procesamiento costoso se ejecuta de forma asíncrona o en isolates.	Obligatorio
RNF-REN-04	Uso responsable de memoria: imágenes decodificadas a un tamaño acorde a su contenedor y caché de imágenes acotada.	Obligatorio
RNF-REN-05	Evidencia de rendimiento del desplazamiento del catálogo y de la navegación durante la sincronización, medida con Flutter DevTools en modo profile.	Recomendado

10.2 Resiliencia

CÓDIGO	REQUISITO	NIVEL
RNF-RES-01	Timeout explícito en las solicitudes remotas, con valor documentado.	Obligatorio
RNF-RES-02	Reintentos con backoff exponencial y límite máximo de intentos.	Obligatorio
RNF-RES-03	Prevención de sincronizaciones concurrentes duplicadas sobre el mismo recurso.	Obligatorio
RNF-RES-04	Escrituras transaccionales: una interrupción no deja datos parciales o inconsistentes visibles.	Obligatorio
RNF-RES-05	Manejo diferenciado de pérdida de conectividad, timeout, errores GraphQL (incluidas respuestas parciales) y limitación de tasa.	Obligatorio
RNF-RES-06	Recuperación de errores sin reiniciar la aplicación.	Obligatorio

10.3 Mantenibilidad

CÓDIGO	REQUISITO	NIVEL
RNF-MAN-01	Código organizado por funcionalidades, con separación por capas.	Obligatorio
RNF-MAN-02	Bajo acoplamiento: el dominio no depende de Flutter, del cliente GraphQL ni de la persistencia.	Obligatorio
RNF-MAN-03	Dependencias inyectadas y sustituibles.	Obligatorio
RNF-MAN-04	Cada dependencia externa relevante está justificada en el README.	Obligatorio

10.4 Accesibilidad

CÓDIGO	REQUISITO	NIVEL
RNF-ACC-01	Etiquetas Semantics en elementos interactivos, imágenes significativas e indicador de conectividad.	Obligatorio
RNF-ACC-02	Contraste mínimo de 4.5:1 para texto normal y 3:1 para texto grande (WCAG 2.1 AA).	Obligatorio
RNF-ACC-03	Objetivos táctiles de al menos 48 × 48 dp.	Obligatorio
RNF-ACC-04	Interfaz utilizable con escalado de texto del sistema al 200% sin pérdida de contenido ni funcionalidad.	Obligatorio
RNF-ACC-05	La información de estado no se transmite exclusivamente mediante color.	Obligatorio

10.5 Calidad

CÓDIGO	REQUISITO	NIVEL
RNF-CAL-01	flutter analyze sin errores ni advertencias, con un conjunto de lints configurado (flutter_lints o equivalente).	Obligatorio
RNF-CAL-02	Código formateado con dart format.	Obligatorio
RNF-CAL-03	Modelo de errores consistente: los fallos de infraestructura se traducen a tipos de fallo del dominio.	Obligatorio
RNF-CAL-04	Código legible y documentado en los puntos no evidentes.	Obligatorio

11 Persistencia y sincronización

Cada equipo debe definir, justificar y documentar las siguientes decisiones. El documento no prescribe una solución; se evalúa la coherencia entre la decisión, su justificación y el comportamiento demostrado.

CÓDIGO	DECISIÓN	DEBE DOCUMENTAR
PS-01	**Modelo de datos local**	Entidades, claves, relaciones e índices; relación con el modelo conceptual (Diagrama 6).
PS-02	**Carga inicial**	Qué datos se descargan en la primera ejecución y cuáles bajo demanda; estrategia de disponibilidad local.
PS-03	**Política de actualización**	Criterios de vigencia de los datos y cuándo se consideran desactualizados.
PS-04	**Sincronización automática y manual**	Eventos que la disparan (inicio, reconexión, intervalo) y mecanismo de actualización manual por el usuario.
PS-05	**Identificación de registros**	Claves estables utilizadas para identificar cada entidad entre sincronizaciones.
PS-06	**Prevención de duplicados**	Mecanismo de inserción o actualización (upsert) y restricciones de unicidad.
PS-07	**Sincronizaciones incompletas**	Comportamiento ante interrupciones; reanudación o reinicio de la operación.
PS-08	**Persistencia de imágenes**	Qué imágenes se almacenan, dónde, con qué límite de espacio y política de limpieza.
PS-09	**Registro de última sincronización**	Metadatos por recurso y cómo se exponen al indicador global (IND-04).
PS-10	**Recuperación ante fallos**	Política de reintentos, estados de error persistidos y migraciones del esquema local.
PS-11	**Separación de datos de usuario**	Garantía de que favoritos, puntuaciones y logros no se ven afectados por la sincronización del catálogo.
PS-12	**Detección de alteraciones locales**	Mecanismo para detectar diferencias entre los datos locales del catálogo y PokeAPI (por ejemplo, comparación por campos, hash o sobrescritura en cada actualización), y frecuencia de la comprobación (OF-19, OF-20).

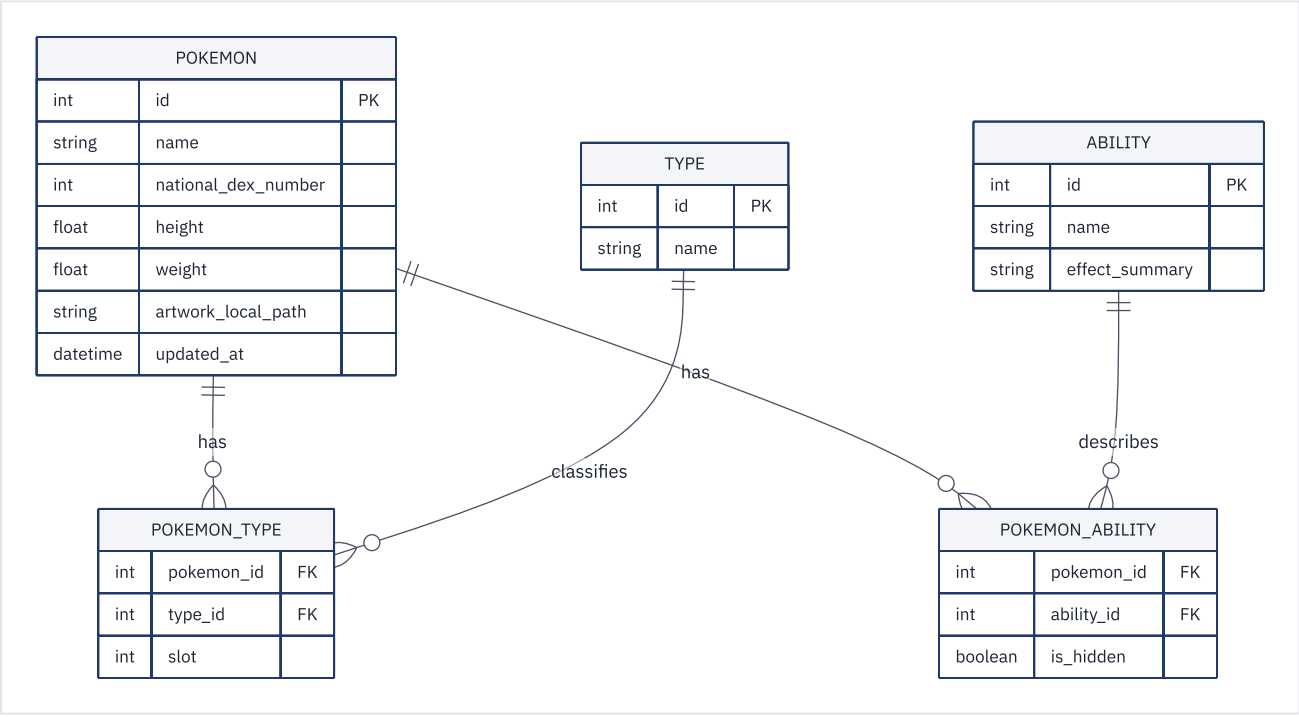
11.1 Aclaraciones

- PokeAPI es una fuente remota principalmente de lectura y es la autoridad de los datos del catálogo. La sincronización remoto → local es obligatoria (OF-19); no se exige sincronización bidireccional.
- Los favoritos, puntuaciones y logros pueden permanecer exclusivamente en almacenamiento local.
- La sincronización incremental solo es obligatoria cuando el servicio remoto ofrezca mecanismos que la hagan viable. En caso contrario, se aceptan estrategias eficientes de actualización justificadas (por ejemplo, por lotes, por recurso o por antigüedad del dato).

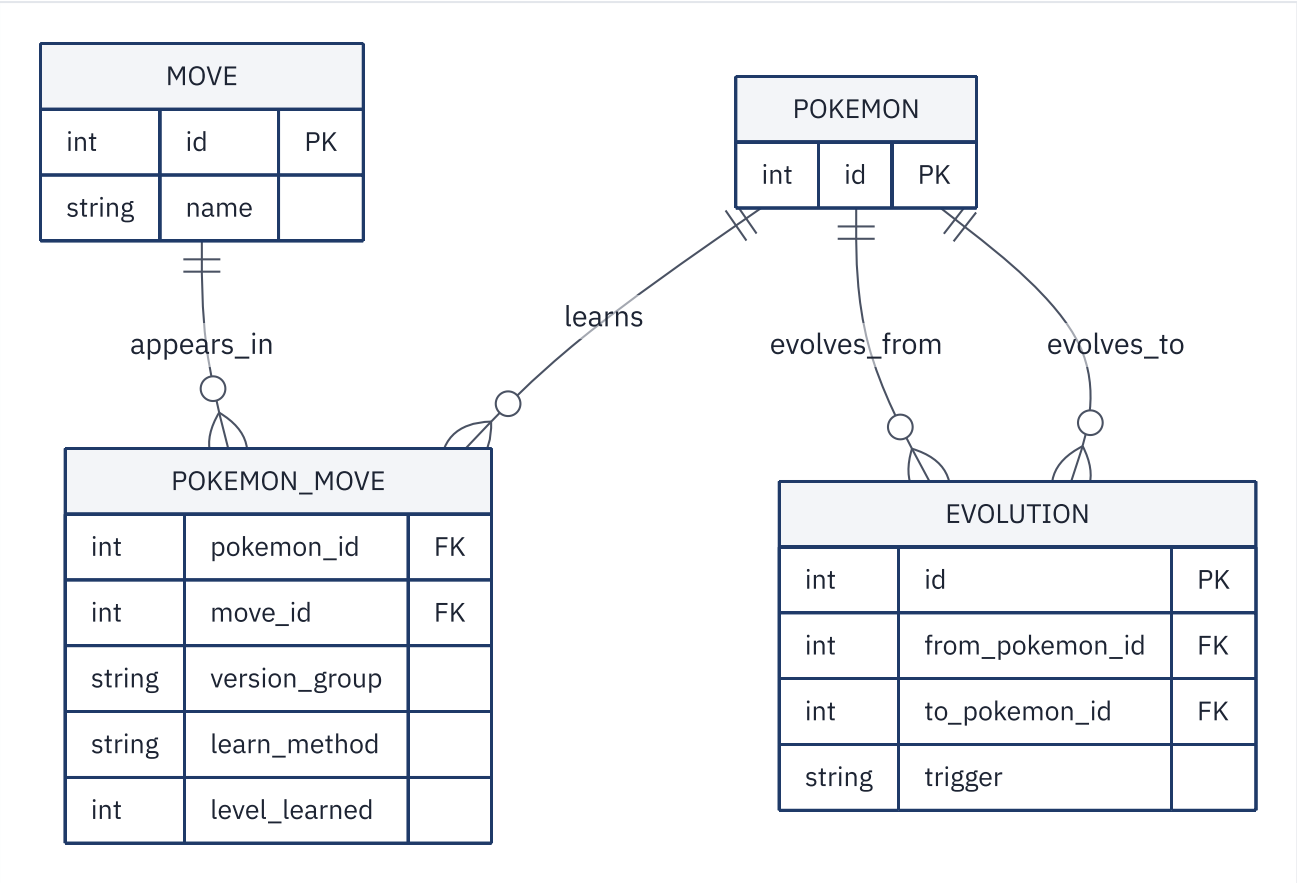
11.2 Modelo conceptual de datos

El Diagrama 6 muestra entidades representativas y sus relaciones conceptuales. No impone un esquema físico definitivo: los equipos pueden ajustar cardinalidades y normalización con justificación técnica.

6.1 CATÁLOGO: TIPOS Y HABILIDADES



6.2 CATÁLOGO: MOVIMIENTOS Y EVOLUCIONES



6.3 DATOS DEL USUARIO Y METADATOS DE SINCRONIZACIÓN (LOCALES)

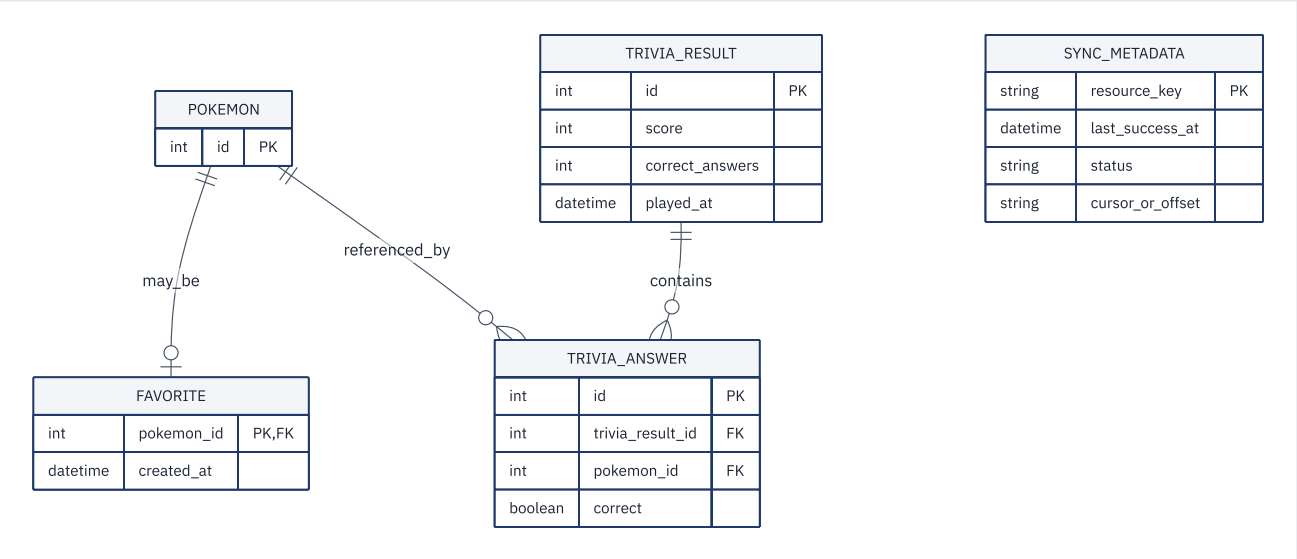


Diagrama 6 — Modelo conceptual de datos. Las evoluciones se representan como relaciones entre especies; las variantes y formas pueden requerir entidades adicionales. `SYNC_METADATA` describe el estado de actualización por recurso y no presupone soporte de cursores por parte de la API. Favoritos y resultados de trivia son datos locales.

12 Diagramas de arquitectura y comportamiento

Los diagramas son modelos conceptuales de referencia y no constituyen una implementación prescrita. Se ubican junto a la sección que explican; esta sección los relaciona con los requisitos y escenarios de aceptación que representan.

ID	DIAGRAMA	SECCIÓN	REQUISITOS Y ESCENARIOS
D1	Flujo Online / Offline	§6.6	OF-01–15 · PA-01, 03, 04, 07, 08, 15
D2	Secuencia de consulta y recuperación	§6.6	OF-02, 04, 16 · RNF-RES-01–05 · PA-02, 08, 09, 13
D3	Máquinas de estados de conectividad y sincronización	§7.4	IND-01–07 · RNF-RES-02, 03 · PA-07, 08, 12, 14
D4	Arquitectura general	§8.1	OF-16–18 · RNF-MAN-01–03 · IND-02
D5	Estructura del proyecto	§8.2	RNF-MAN-01, 02 · ENT-05
D6	Modelo conceptual de datos	§11.2	PS-01, 05, 06, 10 · RF-FAV · RF-TRV · PA-13

12.1 Criterios de revisión arquitectónica

1. La interfaz observa datos persistidos localmente y no depende directamente de respuestas remotas.
2. La aplicación prioriza actualizarse desde GraphQL cuando el servicio está disponible, sin bloquear la navegación.
3. El indicador global diferencia disponibilidad del servicio, actividad de sincronización y procedencia o frescura de los datos.
4. La pérdida de Internet, los timeouts y las interrupciones de sincronización no eliminan información local válida.
5. Los favoritos y la trivia continúan operativos sin conexión cuando sus datos están disponibles.

Los diagramas de la documentación arquitectónica entregada por cada equipo (ENT-07) deben reflejar su solución concreta, no reproducir los de este enunciado.

13 Pruebas de aceptación

Todos los escenarios se demuestran en vivo durante la presentación, en el dispositivo o emulador del equipo. No se aceptan videos ni grabaciones como evidencia. El docente selecciona al azar, en el momento, los escenarios que se ejecutarán, por lo que el equipo debe tenerlos todos preparados. Para PA-08, el equipo debe proveer un mecanismo documentado para simular la indisponibilidad de la API con red activa (por ejemplo, un endpoint configurable o una opción de depuración); para PA-16, un inspector de base de datos accesible durante la demostración.

PA-01 Primera ejecución con Internet

PRECONDICIONES
Aplicación recién instalada, sin datos locales; red y API disponibles.

ACCIÓN
Abrir la aplicación y navegar de inmediato, sin esperar a que termine la carga inicial.

RESULTADO ESPERADO
La interfaz aparece de inmediato y responde a la navegación. El indicador muestra Sincronizando y luego Online; los datos se incorporan progresivamente desde la base local y se registra la última sincronización.

VERIFICACIÓN EN CLASE
Instalación limpia en el dispositivo de demostración; el docente navega antes de que termine la carga.

PA-02 Consulta de Pokémon online

PRECONDICIONES
API disponible; Pokémon no almacenado localmente.

ACCIÓN
Abrir el detalle del Pokémon.

RESULTADO ESPERADO
Los datos se obtienen mediante GraphQL, se persisten y la vista se actualiza desde la base local sin bloquear la navegación.

VERIFICACIÓN EN CLASE
Demostración en vivo; registro de la consulta GraphQL en consola.

PA-03 Consulta de Pokémon almacenados sin Internet

PRECONDICIONES
Pokémon consultados previamente; conexión desactivada.

ACCIÓN
Abrir el catálogo y el detalle de un Pokémon almacenado.

RESULTADO ESPERADO
Se muestran datos e imágenes almacenados; el indicador muestra Offline, el origen local y la última actualización.

VERIFICACIÓN EN CLASE
Demostración en vivo con el estado de conexión del dispositivo visible.

PA-04 Reinicio completo en modo offline

PRECONDICIONES
Datos almacenados; conexión desactivada.

ACCIÓN
Forzar el cierre de la aplicación y volver a abrirla.

RESULTADO ESPERADO
La aplicación inicia sin errores y presenta datos locales, favoritos y preferencias.

VERIFICACIÓN EN CLASE
Cierre forzado y reapertura en vivo.

PA-05 Búsqueda y filtrado sin conexión

PRECONDICIONES
Catálogo parcial o completo almacenado; conexión desactivada.

ACCIÓN
Buscar por nombre, filtrar por tipo y generación y cambiar la ordenación.

RESULTADO ESPERADO
Resultados correctos sobre los datos locales; si el catálogo es parcial, la interfaz lo indica.

VERIFICACIÓN EN CLASE
Demostración en vivo.

PA-06 Agregar y eliminar favoritos sin conexión

PRECONDICIONES
Conexión desactivada.

ACCIÓN
Agregar y eliminar favoritos, abrir la pantalla de favoritos y reiniciar la aplicación.

RESULTADO ESPERADO
Los cambios persisten y se reflejan de forma reactiva en todas las vistas.

VERIFICACIÓN EN CLASE
Demostración en vivo, incluido el reinicio.

PA-07 Recuperación automática de conectividad

PRECONDICIONES
Aplicación en estado Offline con datos locales.

ACCIÓN
Restablecer la conexión sin interactuar con la aplicación.

RESULTADO ESPERADO
El indicador pasa a Sincronizando y luego a Online sin acción del usuario; los datos se actualizan y la navegación continúa.

VERIFICACIÓN EN CLASE
Restablecimiento de la red en vivo, sin interactuar con la aplicación.

PA-08 Fallo de GraphQL con red disponible

PRECONDICIONES
Red disponible; API inaccesible mediante el mecanismo de simulación documentado.

ACCIÓN
Abrir el catálogo y solicitar una actualización.

RESULTADO ESPERADO
El indicador muestra Servicio no disponible, nunca Online. Los datos locales se conservan y los reintentos siguen la política de backoff.

VERIFICACIÓN EN CLASE
Activación en vivo del mecanismo de simulación; registro de reintentos en consola.

PA-09 Interrupción durante la sincronización

PRECONDICIONES
Sincronización en curso.

ACCIÓN
Desactivar la conexión o cerrar la aplicación durante la sincronización; luego restablecer.

RESULTADO ESPERADO
No quedan datos parciales visibles ni corruptos; la sincronización se reanuda o reinicia correctamente y los metadatos reflejan su estado.

VERIFICACIÓN EN CLASE
Interrupción en vivo; inspección de los metadatos de sincronización antes y después.

PA-10 Persistencia y visualización offline de imágenes

PRECONDICIONES
Imágenes de Pokémon consultados previamente; conexión desactivada.

ACCIÓN
Reiniciar la aplicación y abrir catálogo y detalle.

RESULTADO ESPERADO
Las imágenes almacenadas se muestran; las no almacenadas se sustituyen por un recurso de respaldo sin errores.

VERIFICACIÓN EN CLASE
Reinicio en vivo sin conexión.

PA-11 Ejecución de trivia offline

PRECONDICIONES
Datos locales suficientes según el umbral del equipo; conexión desactivada.

ACCIÓN
Jugar una partida completa.

RESULTADO ESPERADO
Silueta, puntuación, tiempo, ranking y logros funcionan; el resultado se persiste. Con datos insuficientes, se muestra un mensaje explicativo.

VERIFICACIÓN EN CLASE
Partida completa en vivo.

PA-12 Actualización correcta del indicador global

PRECONDICIONES
Aplicación en una pantalla principal.

ACCIÓN
Provocar los cuatro estados: red activa, red desactivada, API no disponible y sincronización.

RESULTADO ESPERADO
El indicador refleja cada estado en todas las pantallas principales, con texto e icono, sin bloquear la interacción.

VERIFICACIÓN EN CLASE
El docente provoca los cuatro estados en al menos dos pantallas.

PA-13 Ausencia de duplicados tras sincronizaciones repetidas

PRECONDICIONES
Datos sincronizados.

ACCIÓN
Ejecutar la sincronización completa al menos tres veces consecutivas.

RESULTADO ESPERADO
El número de registros por entidad no cambia; las claves se mantienen estables.

VERIFICACIÓN EN CLASE
Conteo de registros por entidad antes y después, con un inspector de base de datos.

PA-14 Navegación fluida durante sincronización

PRECONDICIONES
Sincronización en curso.

ACCIÓN
Desplazar el catálogo, abrir detalles y cambiar de pantalla.

RESULTADO ESPERADO
La interacción responde sin bloqueos ni saltos perceptibles.

VERIFICACIÓN EN CLASE
Demostración en vivo.

PA-15 Estados vacíos sin datos locales

PRECONDICIONES
Instalación nueva sin datos; conexión desactivada.

ACCIÓN
Abrir catálogo, favoritos y trivia.

RESULTADO ESPERADO
Cada pantalla muestra un estado vacío explicativo con una acción sugerida; no hay errores ni pantallas en blanco.

VERIFICACIÓN EN CLASE
Instalación limpia sin conexión, en vivo.

PA-16	Restauración de datos del catálogo alterados localmente

PRECONDICIONES

Pokémon sincronizado; red y API disponibles.

ACCIÓN

Modificar en la base local un dato del catálogo (por ejemplo, el nombre o un tipo) con un inspector de base de datos; luego abrir el detalle o esperar la siguiente comprobación.

RESULTADO ESPERADO

La aplicación detecta la diferencia y restaura el dato con la versión de PokeAPI; la interfaz se actualiza de forma reactiva. Favoritos y trivia no se ven afectados.

VERIFICACIÓN EN CLASE

Modificación y restauración en vivo con un inspector de base de datos.

14 Entregables y documentación

14.1 Repositorio

ENT-01	Repositorio GitHub público, o privado compartido con el docente.
ENT-02	Historial de commits que refleje el desarrollo incremental del proyecto.
ENT-03	Participación verificable de ambos integrantes mediante commits, pull requests o issues.
ENT-04	Organización del código coherente con la arquitectura documentada.

14.2 README.md ENT-05

El archivo README.md debe incluir, como mínimo:

1. Descripción del proyecto.
2. Requisitos de ejecución.
3. Instrucciones de instalación.
4. Tecnologías utilizadas y su justificación.
5. Arquitectura.
6. Estructura del proyecto.
7. Estrategia Offline-First.
8. Uso de GraphQL y estrategia de paginación.
9. Estrategia de sincronización.
10. Persistencia local.
11. Gestión de conectividad.
12. Funcionalidades adicionales implementadas.
13. Limitaciones conocidas.

14.3 Documentación arquitectónica

ENT-06	Diagrama de arquitectura de la solución implementada.
ENT-07	Modelo de datos local implementado.
ENT-08	Flujo de sincronización implementado.
ENT-09	Justificación tecnológica de las dependencias principales.
ENT-10	Al menos dos Architecture Decision Records (ADR) con contexto, decisión, alternativas consideradas y consecuencias.

14.4 Evidencias

ENT-11	Capturas de pantalla de las pantallas principales y del indicador en sus cuatro estados.

15 Presentación y defensa técnica

La presentación es presencial y obligatoria, dura 15 minutos y requiere la participación de ambos integrantes en las dos partes.

5 min	10 min
Dominio técnico del código · eliminatorio	**Demostración en vivo y preguntas**

15.1 Dominio técnico del código (primeros 5 minutos)

El uso de herramientas de IA está permitido, pero el equipo es responsable de todo el código entregado y debe demostrar que lo comprende y lo controla. El objetivo es la competencia técnica sobre lo construido, por encima de la cantidad de funcionalidades o del acabado visual. Se recomienda no incorporar soluciones que el equipo no domine.

Durante estos 5 minutos, el docente podrá pedir al equipo:

- Explicar cualquier archivo, clase o fragmento de código que el docente elija.
- Describir la responsabilidad de cada capa y el recorrido de un dato desde PokeAPI hasta la interfaz.
- Explicar el flujo de sincronización, la detección de alteraciones locales (OF-19) y el manejo de errores.
- Justificar las decisiones de arquitectura y las dependencias elegidas.
- Indicar dónde y cómo modificaría el código ante un cambio de requisito planteado en el momento.

CRITERIO ELIMINATORIO

Si en estos 5 minutos el equipo no demuestra dominio técnico del código entregado, el proyecto se considera reprobado. La decisión se aplica al grupo completo y la presentación no continúa con la demostración.

15.2 Demostración en vivo (10 minutos)

El docente selecciona al azar escenarios de aceptación (§13) para ejecutarlos en vivo. Además, podrá solicitar:

- Desactivar Internet.
- Reiniciar la aplicación.
- Consultar datos offline.
- Agregar favoritos sin conexión.
- Recuperar Internet.
- Mostrar la sincronización.
- Modificar un dato del catálogo en la base local.

- Explicar decisiones arquitectónicas.

El equipo debe disponer de un dispositivo o emulador con datos previamente sincronizados, el mecanismo de simulación de PA-08 y un inspector de base de datos.

16 Rúbrica de evaluación

La evaluación totaliza 100 puntos. Cada criterio se califica según descriptores observables en cuatro niveles de desempeño, a partir de la demostración, el repositorio, la documentación y la defensa.

#	CRITERIO	PONDERACIÓN	PUNTOS
1	Arquitectura Offline-First y sincronización	30%	30
2	Arquitectura limpia y gestión de estado	25%	25
3	GraphQL e integración de datos	15%	15
4	UI/UX y navegación	10%	10
5	Funcionalidades de la Pokédex	10%	10
6	Documentación y defensa técnica	5%	5
7	Gamificación y creatividad	5%	5
	Total	**100%**	**100**

1. Arquitectura Offline-First y sincronización — 30 pts

Referencias: §6, §11 · PA-01 a PA-16

EXCELENTE · 27–30	BUENO · 21–26	BÁSICO · 15–20	INSUFICIENTE · 0–14
La UI observa exclusivamente la base local. Se superan PA-03 a PA-11, PA-13 y PA-16; reintentos con backoff, control de concurrencia y escrituras transaccionales demostrados y documentados en ADR.	La base local es la fuente de verdad. Se superan los escenarios offline y de reconexión; falla como máximo uno de PA-09, PA-13 o PA-14, sin pérdida de datos.	Funciona offline con datos consultados, pero la reconexión requiere acción manual, o se producen duplicados o bloqueos durante la sincronización.	La UI depende de respuestas remotas para mostrar contenido; sin conexión se pierden datos o la aplicación falla al reiniciarse.

2. Arquitectura limpia y gestión de estado — 25 pts

Referencias: §8, RNF-MAN

EXCELENTE · 23–25	BUENO · 18–22	BÁSICO · 13–17	INSUFICIENTE · 0–12
Capas por funcionalidad; dominio sin dependencias de infraestructura; dependencias inyectadas; estados de carga, éxito, error y vacío modelados explícitamente.	Separación correcta con acoplamientos puntuales justificados; estados explícitos en la mayoría de las pantallas.	Capas nominales con lógica de negocio en widgets o dominio dependiente de infraestructura; estados incompletos.	Sin separación de responsabilidades; los widgets acceden directamente al cliente GraphQL o a la base de datos.

3. GraphQL e integración de datos — 15 pts

Referencias: OF-01, RF-CAT-02, RNF-RES

EXCELENTE · 14–15	BUENO · 11–13	BÁSICO · 8–10	INSUFICIENTE · 0–7
Consultas que solicitan solo los campos necesarios; paginación documentada; DTOs y mapeadores; manejo diferenciado de timeout, red, errores GraphQL y limitación de tasa.	Consultas correctas y paginadas, con mapeo a entidades; manejo de errores general sin diferenciar todos los tipos.	Consultas funcionales con sobrecarga de campos o sin paginación; manejo de errores mínimo.	Integración incompleta o con datos simulados; los errores remotos provocan fallos de la aplicación.

4. UI/UX y navegación — 10 pts

Referencias: §7, RF-CAT-04, RF-CAT-06, RNF-ACC

EXCELENTE · 9–10	BUENO · 7–8	BÁSICO · 5–6	INSUFICIENTE · 0–4
Diseño visual consistente y responsivo; indicador conforme a IND-01 a IND-07; estados vacíos y de error útiles; RNF-ACC verificado.	Interfaz consistente e indicador funcional; deficiencias menores de accesibilidad o de estados.	Interfaz funcional con inconsistencias; indicador incompleto (falta un estado o depende solo del color).	Sin indicador global, navegación con errores o ausencia de estados de error y vacío.

5. Funcionalidades de la Pokédex — 10 pts

Referencias: RF-CAT, RF-DET, RF-FAV, RF-FIL, RF-COM

EXCELENTE · 9–10	BUENO · 7–8	BÁSICO · 5–6	INSUFICIENTE · 0–4
Todos los RF obligatorios de catálogo, detalle, favoritos, filtros y complementarias cumplen sus criterios.	Al menos el 90% de los RF obligatorios; faltantes menores en el detalle.	Entre el 80% y el 89% de los RF obligatorios.	Menos del 80% de los RF obligatorios.

6. Documentación y defensa técnica — 5 pts

Referencias: §14, §15

EXCELENTE · 5	BUENO · 4	BÁSICO · 3	INSUFICIENTE · 0–2
README completo (ENT-05), al menos dos ADR y diagramas consistentes con el código; ambos integrantes responden con precisión.	Documentación completa con omisiones menores; defensa correcta con imprecisiones puntuales.	README incompleto o sin ADR; la defensa recae principalmente en un integrante.	Sin documentación técnica o sin capacidad de explicar la solución.

7. Gamificación y creatividad — 5 pts

Referencias: RF-TRV, RF-COM-05 a RF-COM-08

EXCELENTE · 5	BUENO · 4	BÁSICO · 3	INSUFICIENTE · 0–2
Trivia completa, bilingüe y offline, con logros documentados, y al menos una funcionalidad recomendada u opcional bien integrada.	Trivia completa; funcionalidades recomendadas ausentes o básicas.	Trivia con elementos faltantes (ranking, logros o tiempo límite).	Trivia ausente o no funcional.

17 Criterios de cumplimiento

1. **Cumplimiento mínimo.** El proyecto debe satisfacer al menos el 80% de los requisitos obligatorios definidos en las secciones 6, 7, 9, 10 y 14, verificados mediante demostración o evidencia.
2. **Dominio técnico.** Superar el filtro eliminatorio de los primeros 5 minutos de la presentación (§15.1) es condición previa para cualquier evaluación.
3. **Requisitos indispensables.** Con independencia del porcentaje anterior, el proyecto debe cumplir: sincronización remoto → local (OF-19 a OF-21), interfaz sin bloqueos (OF-03), funcionamiento offline tras reinicio (PA-04), indicador global de conectividad (IND-01 a

IND-05), persistencia local como fuente de verdad (OF-16, OF-17), repositorio con historial de contribuciones (ENT-01 a ENT-03) y presentación presencial con ambos integrantes.

4. **Requisitos recomendados y opcionales.** Su ausencia no afecta el cumplimiento mínimo. Su implementación correcta se valora en los criterios correspondientes de la rúbrica.
5. **Evaluación individual.** La calificación individual puede diferir de la grupal en función de las contribuciones verificables en el repositorio y del desempeño de cada integrante en la defensa técnica.
6. **Autoría.** Cada integrante debe poder explicar cualquier parte del código entregado. El uso de código de terceros o generado con herramientas de IA debe declararse en el README.

18 Referencias técnicas

PokeAPI — documentación	https://pokeapi.co/docs/graphql
PokeAPI — consola GraphQL	https://graphql.pokeapi.co/v1beta2/console
Flutter — guía de arquitectura de aplicaciones	https://docs.flutter.dev/app-architecture
Flutter — patrón Offline-First	https://docs.flutter.dev/app-architecture/design-patterns/offline-first
Flutter — pruebas	https://docs.flutter.dev/testing/overview
Flutter — accesibilidad	https://docs.flutter.dev/ui/accessibility-and-internationalization/accessibility
Flutter — internacionalización	https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization
GraphQL — especificación	https://spec.graphql.org
Riverpod	https://riverpod.dev
BLoC	https://bloclibrary.dev
Drift	https://drift.simonbinder.eu
Material Design 3	https://m3.material.io
WCAG 2.1	https://www.w3.org/TR/WCAG21/
Architecture Decision Records	https://adr.github.io